

Docling Learning Track & Architecture Roadmap

Yes. In fact, now that you've stepped into Docling, I would slightly change our learning path.

Your original OCR path is useful for understanding fundamentals, but for a production-grade RAG, you don't want to spend the entire ingestion journey building OCR pipelines manually. Docling gives us a much broader document-understanding capability.

Think of it this way:

```
OCR
└─ "Can I read the text?"

Docling
└─ "Can I understand the document?"
```

And that is much more interesting for your RAG journey.

What else should we learn from the Docling perspective?

I would create a dedicated Docling Learning Track inside Stage 1.4.2.

We have already completed:

```
Stage 1.4.2.5.2
Docling for Scanned PDFs          [OK]

Stage 1.4.2.5.3
Docling Layout & Reading Order    [OK]
```

From here, I recommend learning these areas.

1. Docling Document Model □ □ □ □ □

This should be our next Docling topic.

We need to understand what this object actually represents:

```
doc = result.document
```

Instead of treating it as just another extracted-text object, we'll learn:

```
DoclingDocument
├─
├─ Text
├─ Headings
├─ Paragraphs
├─ Tables
├─ Pictures
├─ Captions
├─ Lists
├─ Pages
├─ Groups
├─ References
└─ Structure
```

Most importantly: How does Docling represent the relationship between these elements?

This is fundamental for advanced RAG.

2. Docling Export Formats

We've already touched this. We should properly learn:

```
DoclingDocument
├── Markdown
├── Dictionary
├── JSON representation
├── HTML
└── other supported representations
```

And understand:

When should we use Markdown?

For example:

```
PDF
↓
Docling
↓
Markdown
↓
Chunking
↓
Embedding
```

When should we use structured representation?

For example:

```
PDF
↓
Docling
↓
Structured document
↓
Element-aware processing
↓
Chunking
```

This distinction becomes very important in production RAG.

3. Tables — Deep Dive □ □ □ □ □

This is a must learn for your RAG project.

We shouldn't stop at: "Docling can extract tables."

We should learn:

```
PDF Table
↓
Docling
↓
Table structure
↓
Rows / Columns / Cells
↓
Structured representation
```

And then ask: How should tables be represented for RAG?

For example:

Product	Region	Revenue
A	India	10M
B	USA	20M

Should we embed the raw Markdown table?

Or convert it into:

```
Product A
Region: India
Revenue: 10M
```

Or create a table-specific retrieval strategy? That becomes an advanced RAG topic.

4. Pictures and Images

Docling can identify pictures in documents. We should learn:

```

PDF
↓
Docling
↓
Picture detected
↓
Picture metadata / location
↓
Image processing

```

Then we can ask: What do we do when a PDF contains an architecture diagram?

This connects directly to your multimodal RAG goal.

For example:

```

Architecture Diagram
↓
Docling detects picture
↓
Image extraction
↓
Vision model
↓
Description
↓
Metadata / searchable representation

```

5. Architecture Diagrams and Figures 🟡 🟡 🟡 🟡 🟡

This deserves its own experiment.

For example:

```

User
↓
API
↓
Event Hub
↓
Processing
↓
Data Explorer

```

Docling may identify the diagram as a picture, but: Identifying an image and understanding an image are two different problems.

We should learn that distinction.

```

Document understanding
↓
"There's a picture here."

Vision understanding
↓
"This picture represents an Azure Event Hub
architecture."

```

That distinction is crucial for multimodal RAG.

6. Mathematical Formulas

This is another important Docling capability to investigate.

For example:

$$E = mc^2$$

or:

$$P(A|B) = P(B|A)P(A) / P(B)$$

We should learn:

```
PDF
↓
Formula
↓
Document parser
↓
Formula representation
↓
Markdown / LaTeX
```

This is particularly useful for scientific papers, engineering documents, technical manuals, and research documents.

7. Lists and Nested Structures

Consider:

- 1. Authentication
 - 1.1 OAuth
 - 1.2 JWT
- 2. Authorization
 - 2.1 RBAC
 - 2.2 ABAC

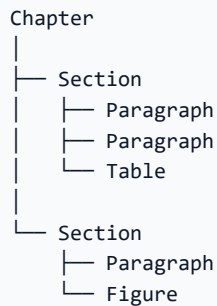
A basic text extractor might flatten this. Docling's document structure gives us an opportunity to preserve:

```
Parent
├─ Child
└─ Child
```

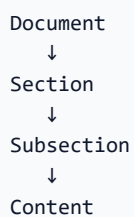
This matters when creating hierarchical chunks.

8. Document Hierarchy □ □ □ □ □

This is one of the most important topics for your RAG learning.
Consider:



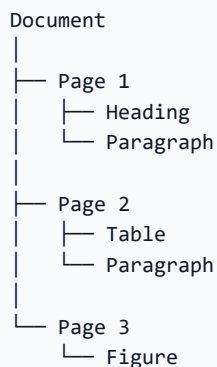
Instead of treating the PDF as one giant text stream, we can preserve:



This leads directly to Hierarchical RAG and Parent-child retrieval.

9. Page-Level Information

We should learn how Docling associates content with pages.
For example:



Then we can attach metadata:

```
{
  "page": 3,
  "document": "architecture.pdf",
  "element_type": "picture"
}
```

This becomes extremely valuable for citations, source references, debugging, retrieval, and UI document previews.

10. Bounding Boxes and Coordinates

This is a more advanced but very useful topic. A document element can have a physical location:



We can potentially associate an element with: x, y, width, height, page.

Why do we care? Because production applications may need: "Show me the exact location of the answer in the original PDF."

11. Metadata Extraction

We should learn how to preserve metadata such as:

```
Document
├──
├── filename
├── page
├── section
├── element type
├── heading hierarchy
├── coordinates
└── source
```

Then our eventual RAG chunk could look conceptually like:

```
{
  "text": "...",
  "metadata": {
    "source": "architecture.pdf",
    "page": 4,
    "section": "Event Processing",
    "element_type": "paragraph"
  }
}
```

This connects Docling directly to LangChain Documents.

12. Docling → LangChain

This should definitely be part of our journey. Eventually:

```
PDF
↓
Docling
↓
DoclingDocument
↓
Structured elements
↓
LangChain Document
↓
Chunking
↓
Embedding
↓
Vector DB
```

This is where our Docling learning becomes directly useful to your RAG application.

13. Docling + Chunking □ □ □ □ □

This is where things become really interesting. Instead of:

```
PDF
↓
Extract all text
↓
RecursiveCharacterTextSplitter
```

we can explore:

```
PDF
↓
Docling
↓
Document structure
↓
Semantic elements
↓
Structure-aware chunking
```

For example: Heading + Paragraph + Paragraph + Table can potentially form a much better semantic unit than every 500 characters.

14. Docling + Hybrid Documents

We should eventually test a single PDF containing:

```
Text
+
Table
+
Image
+
Architecture Diagram
+
Formula
+
Scanned Page
```

This is actually very close to the sample PDF you asked me to create earlier. Then we can see how Docling handles each component. That would make an excellent practical exercise.

15. Docling vs Traditional PDF Loaders

We should also perform a proper comparison:

Capability	PyPDFLoader	PyMuPDF	Docling
Text	[OK]	[OK]	[OK]
Pages	[OK]	[OK]	[OK]
Reading order	Limited	Limited	□
Tables	Limited	Limited	□ □ □
Layout	Limited	Limited	□ □ □
Images	Limited	[OK]	□ □ □
Structure	Limited	Limited	□ □ □
Complex PDFs	Limited	Good	□ □ □ □ □
RAG-oriented document understanding	Basic	Basic	Strong

The exact capabilities depend on the document and configuration, so we'll validate them hands-on rather than treating the table as absolute.

16. Docling Configuration

Eventually we'll learn that `converter = DocumentConverter()` is only the beginning. We'll investigate things such as:

```
DocumentConverter
├── PDF pipeline options
├── OCR options
├── table options
├── image options
└── processing configuration
```

This is where we start moving toward production configuration.

17. Docling Performance & Production Considerations

Later, we should investigate:

Large PDFs
Thousands of PDFs
Parallel processing
Caching
OCR cost
Processing time
Memory usage
Failure handling
Logging

Because: A library working perfectly on a 5-page PDF is not automatically a production ingestion pipeline.

How I would restructure your learning journey

I would not abandon OCR, but I would avoid spending too much time manually implementing OCR pipelines now. I'd make the next part:

Stage 1.4.2 – OCR & Complex PDF Ingestion

- | 1.4.2.1 – Text vs Scanned PDF [OK]
- | 1.4.2.2 – Detecting Scanned Pages [OK]
- | 1.4.2.3 – Deep PDF Page Analysis [OK]
- | 1.4.2.4 – OCR Fundamentals [OK]
- | 1.4.2.5 – OCR Approach
- | | 1.4.2.5.1 – Basic OCR [OK]
- | | 1.4.2.5.2 – Docling for Scanned PDFs [OK]
- | | 1.4.2.5.3 – Layout & Reading Order [OK]

Docling Deep Dive

- | Docling Document Model
- | Export & Serialization
- | Tables
- | Pictures & Figures
- | Mathematical Formulas
- | Lists & Hierarchy
- | Page & Element Metadata
- | Bounding Boxes
- | Document Structure
- | Structure-aware Chunking
- | Docling → LangChain Documents
- | Multimodal / Vision Integration
- | Complex Mixed PDFs
- | Docling vs PDF Loaders
- | Production Configuration & Performance

My recommendation for our very next lesson

Don't jump to OCR preprocessing yet.
I'd make our next lesson:

```
Stage 1.4.2.5.4 – Docling Document Model
```

Because we've already successfully created:

```
doc = result.document
```

but we haven't really understood what DoclingDocument contains and how its elements are represented. Once you understand that, everything else—tables, figures, hierarchy, metadata, bounding boxes, structure-aware chunking, and Docling → LangChain—will make much more sense. That would be a much stronger progression for your RAG learning journey.